📚 Руководство для начинающих — Health Dashboard DevOps Project

> **Этот документ** объясняет проект простыми словами на русском языке.
> Если ты новичок в DevOps — начни читать отсюда. Здесь всё по порядку.

📑 Содержание

1. Введение в DevOps
2. Технологии в проекте
3. Структура проекта
4. Как это работает
5. Как запустить проект локально
6. Как демонстрировать проект преподавателю
7. Частые вопросы и ответы
8. Чек-лист перед защитой

---

1. 🌐 Введение в DevOps

Что такое DevOps?

DevOps — это подход к разработке программного обеспечения, который объединяет **разработку** (Development) и **эксплуатацию** (Operations).

Представь себе конвейер на заводе:
- **Разработчик** пишет код (как рабочий собирает деталь)
- **DevOps инженер** строит и обслуживает конвейер, который автоматически проверяет, собирает и доставляет эту деталь заказчику

Простая аналогия: Если программист — это повар, который готовит блюдо, то DevOps инженер — это тот, кто построил кухню, наладил поставки продуктов, установил систему контроля качества и организовал доставку блюд клиентам.

Зачем это нужно?

Без DevOps:
- ❌ Код проверяется вручную → ошибки
- ❌ Деплой делается руками → долго и рискованно
- ❌ Сервера настраиваются вручную → «у меня работало!»
- ❌ Нет мониторинга → узнаём о проблемах от пользователей

С DevOps:
- ✅ Код автоматически тестируется при каждом изменении
- ✅ Деплой происходит автоматически и безопасно

- ✅ Инфраструктура описана как код — воспроизводима
- ✅ Мониторинг показывает проблемы до того, как о них узнает пользователь

Что делает DevOps инженер?

Задача	Описание
CI/CD	Настраивает автоматические пайплайны сборки и деплоя
Контейнеризация	Упаковывает приложения в Docker контейнеры
Оркестрация	Управляет контейнерами через Kubernetes
IaC	Описывает инфраструктуру как код (Terraform)
Мониторинг	Настраивает сбор метрик и алертов
Автоматизация	Автоматизирует рутинные задачи (Ansible)
Безопасность	Следит за безопасностью кода и инфраструктуры

---

2. 🛠️ Технологии в проекте

🐳 Docker (контейнеры)

Что это?
Docker — это инструмент для создания **контейнеров**. Контейнер — это как лёгкая виртуальная машина, которая содержит приложение и всё необходимое для его работы.

Аналогия: Представь контейнер для перевозки грузов на корабле. Неважно, что внутри — телевизоры, одежда или еда — контейнер стандартный, его легко грузить и перевозить. Docker делает то же самое с программами.

Зачем в проекте?
- Наше Flask-приложение упаковано в Docker-контейнер
- Это гарантирует, что приложение будет работать одинаково везде: на твоём ноутбуке, на сервере, в облаке

Как это работает?
1. `Dockerfile` — «рецепт» для создания образа (image)
2. Docker читает `Dockerfile` и создаёт образ
3. Из образа запускается контейнер (работающее приложение)

Где в проекте?
- `Dockerfile` — описание как собрать образ приложения

- Использует multi-stage build (2 этапа сборки) для уменьшения размера
- Запускается от непривилегированного пользователя (безопасность)

```
Dockerfile
├── Stage 1 (builder) → Установка Python-зависимостей
└── Stage 2 (production) → Лёгкий образ с приложением
```

🐳 Docker Compose (оркестрация контейнеров)

Что это?
Docker Compose — инструмент для запуска **нескольких контейнеров** одной командой.

Аналогия: Если Docker — это один музыкант, то Docker Compose — это дирижёр оркестра. Он говорит каждому музыканту (контейнеру) когда начинать и как взаимодействовать с другими.

Зачем в проекте?
Наш проект состоит из 6 сервисов, которые должны работать вместе:
- Flask-приложение
- PostgreSQL (база данных)
- Redis (кэш)
- Nginx (веб-сервер)
- Prometheus (метрики)
- Grafana (дашборды)

Где в проекте?
- `docker-compose.yml` — описание всех сервисов
- Одна команда `docker-compose up -d` запускает ВСЁ

```yaml
# Упрощённая схема docker-compose.yml:
services:
  app:        # Flask-приложение (порт 5000)
  postgres:   # База данных (порт 5432)
  redis:      # Кэш (порт 6379)
  nginx:      # Веб-сервер (порт 80)
  prometheus: # Метрики (порт 9090)
  grafana:    # Дашборды (порт 3000)
```

🌶️ Flask (веб-приложение)

Что это?
Flask — это лёгкий фреймворк для создания веб-приложений на Python.

Аналогия: Flask — это как конструктор LEGO для веб-сайтов. Простой, гибкий, можно собрать что угодно.

Зачем в проекте?
Это **главное приложение** проекта — Health Dashboard. Оно:
- Показывает дашборд с информацией о системе (CPU, RAM, диск)
- Предоставляет API для проверки здоровья системы

- Отдаёт метрики для Prometheus
- Пишет структурированные JSON-логи в stdout контейнера

Где в проекте?
- `app/app.py` — основной код приложения
- `app/wsgi.py` — точка входа для Gunicorn (production-сервер)
- `requirements.txt` — список Python-зависимостей

Основные эндпоинты (URL-адреса):

URL	Что делает
`/`	Главная страница — HTML-дашборд
`/health`	Проверка здоровья приложения (JSON)
`/metrics`	Метрики для Prometheus
`/api/system-info`	Информация о системе (CPU, RAM, диск)

🐘 PostgreSQL (база данных)

Что это?
PostgreSQL — это реляционная база данных. Она хранит данные в таблицах (как Excel, но мощнее).

Аналогия: Если приложение — это магазин, то PostgreSQL — это склад, где хранятся все товары (данные).

Зачем в проекте?
- Хранит исторические метрики системы (CPU, RAM, диск)
- Приложение записывает данные при каждом сборе метрик

Где в проекте?
- Запускается как контейнер через `docker-compose.yml`
- Данные сохраняются в Docker volume `postgres_data`
- Настройки подключения — в переменных окружения (`.env`)

⚡ Redis (кэш)

Что это?
Redis — это хранилище данных в оперативной памяти. Работает очень быстро.

Аналогия: Redis — это как записная книжка на рабочем столе. Она маленькая, но обращаться к ней намного быстрее, чем идти на склад (базу данных).

Зачем в проекте?
- Кэширует результаты системных метрик на 10 секунд
- Уменьшает нагрузку на базу данных
- Ускоряет ответы API

Где в проекте?
- Запускается как контейнер через `docker-compose.yml`
- Данные сохраняются в Docker volume `redis_data`

🌐 Nginx (веб-сервер / reverse proxy)

Что это?
Nginx — это высокопроизводительный веб-сервер, который работает как «входная дверь» для приложения.

Аналогия: Nginx — это как администратор на ресепшене в офисе. Он встречает всех посетителей (запросы) и направляет их к нужному сотруднику (сервису).

Зачем в проекте?
- Принимает HTTP-запросы на порт 80
- Перенаправляет их к Flask-приложению на порт 5000
- Добавляет заголовки безопасности
- Может балансировать нагрузку между несколькими копиями приложения

Где в проекте?
- `nginx/nginx.conf` — конфигурация Nginx
- Запускается как контейнер через `docker-compose.yml`

🏗️ Terraform (инфраструктура как код)

Что это?
Terraform — инструмент для создания облачной инфраструктуры с помощью кода.

Аналогия: Вместо того чтобы вручную кликать кнопки в AWS-консоли, ты описываешь инфраструктуру в файлах. Это как чертёж для строительства — по нему можно построить одинаковый дом сколько угодно раз.

Зачем в проекте?
- Создаёт EC2-сервер в AWS
- Настраивает Security Group (правила файрвола)
- Автоматически устанавливает Docker на сервер

Где в проекте?
- `terraform/main.tf` — основная конфигурация (EC2, Security Group)
- `terraform/variables.tf` — переменные (регион, тип сервера, SSH-ключ)
- `terraform/outputs.tf` — выходные данные (IP-адрес сервера, URL)

Основные команды:

```
terraform init    # Инициализация (скачать провайдеры)
terraform plan    # Показать что будет создано
terraform apply   # Создать инфраструктуру
terraform destroy # Удалить всё
```

☸️ Kubernetes (оркестрация контейнеров)

Что это?
Kubernetes (K8s) — система для автоматического управления контейнерами в масштабе.

Аналогия: Если Docker Compose — это дирижёр маленького оркестра, то Kubernetes — это менеджер огромной филармонии. Он управляет сотнями музыкантов, заменяет заболевших, добавляет новых при необходимости.

Зачем в проекте?
- Показывает как приложение можно развернуть в продакшен
- Автоматический перезапуск упавших контейнеров
- Масштабирование (можно легко добавить копии приложения)
- Балансировка нагрузки

Где в проекте?
- `k8s/namespace.yaml` — пространство имён
- `k8s/configmap.yaml` — конфигурация (не секретная)
- `k8s/secret.yaml` — секреты (пароли)
- `k8s/deployment.yaml` — описание деплоймента (2 реплики)
- `k8s/service.yaml` — сервис (LoadBalancer)
- `k8s/helm/` — Helm-чарт (шаблонизация манифестов)

🤖 Ansible (автоматизация)

Что это?
Ansible — инструмент для автоматической настройки серверов.

Аналогия: Ansible — это как подробная инструкция для нового сотрудника. Вместо того чтобы каждый раз объяснять, как настроить рабочее место, ты даёшь ему чек-лист, и он всё делает сам.

Зачем в проекте?
- Автоматически устанавливает Docker на сервер
- Копирует файлы проекта на сервер
- Запускает приложение через Docker Compose
- Настраивает файрвол

Где в проекте?
- `ansible/inventory.ini` — список серверов
- `ansible/playbook.yml` — основной плейбук (что делать)
- `ansible/roles/docker/` — роль для установки Docker
- `ansible/roles/app/` — роль для деплоя приложения

⚙️ GitHub Actions (CI/CD)

Что это?
GitHub Actions — это сервис автоматизации прямо в GitHub. CI/CD означает:
- **CI** (Continuous Integration) — автоматическая проверка кода
- **CD** (Continuous Delivery/Deployment) — автоматическая доставка кода

Аналогия: Представь, что каждый раз, когда ты кладёшь письмо в почтовый ящик (push в GitHub), автоматически:
1. Робот проверяет, нет ли ошибок в письме (тесты)
2. Другой робот красиво его упаковывает (сборка Docker-образа)
3. Третий робот доставляет его получателю (деплой на сервер)

Зачем в проекте?
- Автоматически запускает тесты при каждом push
- Собирает Docker-образ и загружает его на Docker Hub
- Деплоит приложение на сервер (если настроен)

Где в проекте?
- `.github/workflows/ci-cd.yml` — описание пайплайна

Структура пайплайна:

```
git push → [Тесты ✅] → [Сборка Docker 🐳] → [Деплой 🚀]
                ↓                ↓                  ↓
          pytest + flake8   Push на Docker Hub   SSH на сервер
```

📊 Prometheus (метрики)

Что это?
Prometheus — система мониторинга, которая собирает числовые метрики с приложений.

Аналогия: Prometheus — это как датчики в автомобиле. Они постоянно показывают скорость, температуру двигателя, уровень топлива. Если что-то не так — загорается лампочка.

Зачем в проекте?
- В текущей конфигурации каждые 60 секунд запрашивает метрики у приложения (`/metrics`)
- Хранит историю метрик (CPU, RAM, диск, количество запросов)
- Позволяет настроить алерты (оповещения)

Где в проекте?
- `monitoring/prometheus.yml` — конфигурация Prometheus
- `monitoring/alert_rules.yml` — правила алертов
- Доступен по адресу: `http://localhost:9090`

📈 Grafana (визуализация)

Что это?
Grafana — инструмент для создания красивых дашбордов и графиков.

Аналогия: Если Prometheus — это датчики, то Grafana — это приборная панель автомобиля, где ты видишь все показатели в удобном виде.

Зачем в проекте?
- Визуализирует метрики из Prometheus
- Показывает графики CPU, RAM, диска

- Готовый дашборд «Health Dashboard» уже настроен
- Для логов используется `docker compose logs` (Loki удалён)

Где в проекте?
- `monitoring/grafana/provisioning/` — автоматическая настройка
- `monitoring/grafana/dashboards/health-dashboard.json` — готовый дашборд
- Доступна по адресу: `http://localhost:3000` (логин/пароль из `.env`)

📝 Логирование (без Loki)

Что это?
Приложение пишет структурированные JSON-логи в stdout контейнера.

Аналогия: Это как журнал событий, который всегда можно быстро посмотреть через Docker.

Зачем в проекте?
- Диагностика ошибок приложения
- Анализ поведения API при тестировании и деплое
- Быстрая отладка без дополнительного тяжёлого log-stack

Где в проекте?
- Логи приложения: `docker compose logs app`
- Поток логов в реальном времени: `docker compose logs -f app`
- Loki удалён из оптимизированной конфигурации для `t3.micro`

---

3. 📁 Структура проекта

```
my-devops-project/
│
├── 📂 app/                          # Приложение (Flask)
│   ├── __init__.py                  # Делает папку Python-пакетом
│   ├── app.py                       # Главный файл приложения
│   └── wsgi.py                      # Точка входа для Gunicorn
│
├── 📂 terraform/                    # Инфраструктура как код (AWS)
│   ├── main.tf                      # EC2 сервер + Security Group
│   ├── variables.tf                 # Переменные (регион, тип сервера)
│   └── outputs.tf                   # Выходные данные (IP, URL)
│
├── 📂 k8s/                          # Kubernetes манифесты
│   ├── namespace.yaml               # Пространство имён
│   ├── configmap.yaml               # Конфигурация
│   ├── secret.yaml                  # Секреты (пароли)
│   ├── deployment.yaml              # Деплоймент (2 реплики)
│   ├── service.yaml                 # Сервис (LoadBalancer)
│   └── 📂 helm/                     # Helm-чарт
│       └── 📂 health-dashboard/
│           ├── Chart.yaml           # Метаданные чарта
│           ├── values.yaml          # Значения по умолчанию
│           └── 📂 templates/        # Шаблоны манифестов
│
├── 📂 ansible/                      # Автоматизация серверов
│   ├── inventory.ini                # Список серверов
│   ├── playbook.yml                 # Основной плейбук
│   └── 📂 roles/                    # Роли
│       ├── 📂 docker/tasks/         # Установка Docker
│       └── 📂 app/tasks/            # Деплой приложения
│
├── 📂 monitoring/                   # Мониторинг
│   ├── prometheus.yml               # Конфигурация Prometheus
│   ├── alert_rules.yml              # Правила алертов
│   └── 📂 grafana/                  # Grafana
│       ├── 📂 provisioning/         # Автонастройка
│       │   ├── datasources/         # Источники данных
│       │   └── dashboards/          # Провайдер дашбордов
│       └── 📂 dashboards/           # JSON-дашборды
│
├── 📂 tests/                        # Тесты
│   ├── __init__.py                  # Python-пакет
│   ├── conftest.py                  # Фикстуры pytest
│   ├── test_app.py                  # Тесты приложения
│   └── test_health.py               # Тесты API
│
├── 📂 nginx/                        # Конфигурация Nginx
│   └── nginx.conf                   # Reverse proxy настройки
│
├── 📂 .github/workflows/           # CI/CD пайплайн
│   └── ci-cd.yml                    # GitHub Actions workflow
│
├── 📂 docs/                         # Документация
│   ├── BEGINNER_GUIDE_RU.md         # Это руководство!
│   ├── DEMO_SCRIPT_RU.md            # Сценарий демонстрации
│   ├── ARCHITECTURE.md              # Архитектура проекта
│   ├── DEPLOYMENT.md                # Инструкция по деплою
│   └── ...                          # Другие документы
│
├── Dockerfile                       # Рецепт Docker-образа
├── docker-compose.yml               # Оркестрация 6 сервисов
└── requirements.txt                 # Python-зависимости
```

```
├── Makefile                    # Удобные команды
├── .env.example                # Шаблон переменных окружения
├── .gitignore                  # Файлы, игнорируемые Git
├── README.md                   # Главная документация
├── SECURITY_AUDIT.md           # Отчёт по безопасности
└── CONTRIBUTING.md             # Как вносить изменения
```

---

4. ⚙️ Как это работает

4.1 Что происходит, когда ты делаешь `git push`

```
Ты: git push origin main
        │
        ▼
┌─── GitHub получает код ───┐
│                           │
│  GitHub Actions срабатывает │
│  (.github/workflows/      │
│   ci-cd.yml)              │
│                           │
└───────────────────────────┘
        │
        ▼
┌─── Job 1: ТЕСТЫ ──────────┐
│                           │
│  1. Скачивает код         │
│  2. Ставит Python 3.11    │
│  3. Ставит зависимости (pip) │
│  4. Запускает pytest      │
│  5. Запускает flake8 (линт) │
│                           │
│  ✅ Тесты прошли!         │
└───────────────────────────┘
        │
        ▼
┌─── Job 2: СБОРКА ─────────┐
│                           │
│  1. Логинится в Docker Hub │
│  2. Собирает Docker-образ │
│  3. Пушит образ на Docker Hub│
│     - :latest             │
│     - :sha-коммита        │
│                           │
│  ✅ Образ загружен!       │
└───────────────────────────┘
        │
        ▼
┌─── Job 3: ДЕПЛОЙ ─────────┐
│                           │
│  Если сервер настроен:    │
│  1. SSH на сервер         │
│  2. docker-compose pull   │
│  3. docker-compose up -d  │
│                           │
│  Если нет — пропускается  │
│  ⚠️ Это нормально!        │
└───────────────────────────┘
```

4.2 Как Docker собирает образ

```
Dockerfile
    │
    ├── Stage 1: builder
    │   ├── Берёт образ Python 3.11
    │   ├── Копирует requirements.txt
    │   └── Устанавливает Python-пакеты
    │
    └── Stage 2: production
        ├── Берёт чистый образ Python 3.11
        ├── Создаёт пользователя appuser (безопасность)
        ├── Копирует установленные пакеты из Stage 1
        ├── Копирует код приложения (app/)
        ├── Переключается на appuser
        └── Запускает Gunicorn (production-сервер)

Результат: Лёгкий, безопасный Docker-образ
```

4.3 Как приложение запускается

```
docker-compose up -d
        │
        ├── 1. Запускается PostgreSQL (ждёт готовности)
        ├── 2. Запускается Redis (ждёт готовности)
        ├── 3. Запускается Flask-приложение
        │      ├── Подключается к PostgreSQL
        │      ├── Подключается к Redis
        │      ├── Создаёт таблицу metrics в БД
        │      └── Слушает на порту 5000
        ├── 4. Запускается Nginx (проксирует на порт 5000)
        ├── 5. Запускается Prometheus (парсит /metrics)
        └── 6. Запускается Grafana (визуализирует метрики)
```

4.4 Как работает мониторинг

```
Flask-приложение                    Prometheus
      │                                 │
      │ /metrics (каждые 60 сек)        │
      │◄────────────────────────────────┤
      │                                 │
      │ cpu_usage=45%, mem=60%...       │
      ├────────────────────────────────►│
      │                                 │
      │                                 │    Grafana
      │                                 │       │
      │                                 │ ◄─────┤ (запрашивает данные)
      │                                 │       │
      │                                 │ ──────► (красивые графики)
      │                                 │
      │ JSON-логи ──────► docker compose logs app
```

5. 🚀 Как запустить проект локально

Шаг 1: Установи Docker Desktop

Windows:

1. Скачай с docker.com/products/docker-desktop (https://www.docker.com/products/docker-desktop)
2. Установи (Next → Next → Finish)
3. Перезагрузи компьютер
4. Открой Docker Desktop — подожди пока значок в трее станет зелёным

Mac:

1. Скачай с docker.com/products/docker-desktop (https://www.docker.com/products/docker-desktop)
2. Перетащи в Applications
3. Запусти и подожди

Linux:

```
# Ubuntu/Debian
sudo apt update
sudo apt install docker.io docker-compose
sudo usermod -aG docker $USER
# Перелогинься
```

Проверка установки:

```
docker --version        # Должно показать версию
docker-compose --version # Должно показать версию
```

Шаг 2: Клонируй репозиторий

```
git clone https://github.com/zaburdaev/my-devops-project.git
cd my-devops-project
```

Шаг 3: Настрой .env файл

```
# Скопируй пример
cp .env.example .env

# Можешь оставить значения по умолчанию для локального запуска
# Или отредактируй .env файл
```

Содержимое `.env` для локального запуска (можно не менять):

```
FLASK_DEBUG=false
SECRET_KEY=my-local-secret-key
APP_PORT=5000
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DB=healthdb
POSTGRES_USER=admin
POSTGRES_PASSWORD=changeme
REDIS_HOST=redis
REDIS_PORT=6379
GF_SECURITY_ADMIN_USER=admin
GF_SECURITY_ADMIN_PASSWORD=admin
```

Шаг 4: Запусти проект

```
# Запусти все сервисы
docker-compose up -d

# Или через Makefile:
make up
```

Подожди 30-60 секунд, пока все сервисы запустятся.

Шаг 5: Проверь, что всё работает

```
# Посмотри статус контейнеров
docker-compose ps

# Все контейнеры должны быть в статусе "Up" или "healthy"
```

Ожидаемый результат:

```
NAME                STATUS
app                 Up (healthy)
postgres            Up (healthy)
redis               Up (healthy)
nginx               Up
prometheus          Up
grafana             Up
# Loki удалён в оптимизированной конфигурации
```

Шаг 6: Открой в браузере

Сервис	URL	Описание
🌐 Приложение	http://localhost (или :80)	Health Dashboard
🌐 Приложение (напрямую)	http://localhost:5000	Flask без Nginx
📊 Grafana	http://localhost:3000	Дашборды (логин/пароль из `.env`)
📈 Prometheus	http://localhost:9090	Метрики
❤️ Health check	http://localhost:5000/health	JSON-ответ
📊 Metrics	http://localhost:5000/metrics	Prometheus-метрики
ℹ️ System info	http://localhost:5000/api/system-info	Информация о системе

Шаг 7: Проверь Grafana

1. Открой http://localhost:3000
2. Логин: `admin` / Пароль: `admin`
3. Перейди в **Dashboards → Health Dashboard**
4. Ты увидишь графики CPU, RAM, Disk

Шаг 8: Посмотри логи

```
# Логи всех сервисов
docker-compose logs

# Логи конкретного сервиса
docker-compose logs app
docker-compose logs postgres

# Логи в реальном времени
docker-compose logs -f app
```

---

6. 🎓 Как демонстрировать проект преподавателю

Порядок демонстрации

1. Начни с GitHub-репозитория

> «Вот наш проект на GitHub. Здесь хранится весь код, документация и CI/CD пайплайн.»

- Покажи структуру файлов в GitHub
- Покажи README.md
- Покажи количество коммитов

2. Покажи CI/CD Pipeline

> «При каждом push в main ветку автоматически запускается CI/CD пайплайн.»

- Открой вкладку **Actions** в GitHub
- Покажи последний успешный запуск
- Открой детали — покажи 3 этапа: Test → Build → Deploy
- Объясни: «Тесты прошли, Docker-образ собран и загружен на Docker Hub»

3. Покажи Docker Hub

> «Собранный образ хранится на Docker Hub и доступен для скачивания.»

- Открой https://hub.docker.com/r/oskalibriya/health-dashboard
- Покажи теги (latest, sha-коммита)

4. Покажи работающее приложение

> «Запустим приложение локально с помощью Docker Compose.»

```
docker-compose up -d
docker-compose ps
```

- Открой http://localhost:5000 — покажи дашборд
- Открой http://localhost:5000/health — покажи JSON-ответ
- Открой http://localhost:5000/api/system-info — покажи метрики

5. Покажи мониторинг (Grafana)

> «Вот система мониторинга. Grafana визуализирует метрики, которые собирает Prometheus.»

- Открой http://localhost:3000 (учётные данные из `.env`)
- Покажи дашборд Health Dashboard
- Объясни графики: CPU, Memory, Disk, Request Rate

6. Покажи логи

> «В текущей оптимизированной конфигурации логи читаем напрямую через Docker, формат JSON.»

- Выполни `docker compose logs --tail=50 app`
- Покажи поток `docker compose logs -f app`

7. Покажи Terraform код

> «Инфраструктура описана как код с помощью Terraform.»

- Открой `terraform/main.tf`
- Объясни: «Здесь описано создание EC2-сервера в AWS»
- Покажи `terraform/variables.tf` — «Переменные можно менять»
- Покажи `terraform/outputs.tf` — «После деплоя получаем IP и URL»

8. Покажи Kubernetes манифесты

> «Для production-окружения приложение можно развернуть в Kubernetes.»

- Покажи `k8s/deployment.yaml` — «2 реплики приложения с health-проверками»

- Покажи `k8s/service.yaml` — «LoadBalancer для внешнего доступа»
- Покажи `k8s/helm/` — «Helm-чарт для шаблонизации»

9. Покажи Ansible playbook

> «Ansible автоматизирует настройку сервера и деплой.»

- Покажи `ansible/playbook.yml`
- Объясни: «Устанавливает Docker, копирует файлы, запускает приложение»

10. Покажи тесты

> «Проект покрыт тестами. Они запускаются автоматически в CI/CD.»

```
# Запусти тесты локально
python -m pytest tests/ -v
```

Что говорить при демонстрации

Про архитектуру:

> «Проект использует микросервисную архитектуру. Flask-приложение, база данных PostgreSQL, кэш Redis, веб-сервер Nginx — всё работает в отдельных Docker-контейнерах.»

Про CI/CD:

> «CI/CD пайплайн реализован через GitHub Actions. При каждом push автоматически: прогоняются тесты, собирается Docker-образ, и если всё OK — деплоится на сервер.»

Про мониторинг:

> «Для мониторинга используется стек Prometheus + Grafana. Prometheus собирает метрики, а Grafana визуализирует их. Логи приложения читаются через docker compose logs.»

Про безопасность:

> «Чувствительные данные (пароли, ключи) хранятся в переменных окружения и GitHub Secrets. В коде только placeholder-значения.»

---

7. ❓ Частые вопросы и ответы

Docker не запускается

Проблема: Docker Desktop не стартует или показывает ошибку.

Решения:

```
# Windows: Проверь, что WSL2 включён
wsl --install
wsl --update

# Перезапусти Docker Desktop
# Правый клик на иконку Docker → Restart

# Linux: Проверь статус сервиса
sudo systemctl status docker
sudo systemctl start docker
```

Порты заняты

Проблема: `Error: port is already allocated`

Решения:

```
# Посмотри, что занимает порт (например, 5000)
# Windows:
netstat -ano | findstr :5000

# Linux/Mac:
lsof -i :5000

# Останови другой процесс или измени порт в docker-compose.yml
# Например: ports: "5001:5000" вместо "5000:5000"
```

Тесты падают

Проблема: `pytest` показывает ошибки.

Решения:

```
# Убедись, что зависимости установлены
pip install -r requirements.txt

# Запусти тесты с подробным выводом
python -m pytest tests/ -v --tb=long

# Тесты не требуют БД и Redis — они используют Flask test client
```

Как посмотреть логи контейнеров

```
# Все логи
docker-compose logs

# Логи конкретного сервиса
docker-compose logs app
docker-compose logs postgres
docker-compose logs nginx

# Логи в реальном времени (Ctrl+C для выхода)
docker-compose logs -f

# Последние 50 строк
docker-compose logs --tail=50 app
```

Как остановить все контейнеры

```
# Остановить (данные сохраняются)
docker-compose down

# Остановить и удалить данные (volumes)
docker-compose down -v

# Или через Makefile
make down
```

Как пересобрать образы

```
# Пересобрать без кэша
docker-compose build --no-cache

# Пересобрать и запустить
docker-compose up -d --build

# Или через Makefile
make build
make up
```

Grafana показывает «No data»

Решения:

1. Подожди 1-2 минуты — Prometheus нужно время для сбора метрик
2. Проверь, что приложение работает: `curl http://localhost:5000/health`
3. Проверь Prometheus: http://localhost:9090 → Status → Targets → все должны быть UP
4. В Grafana проверь data source: Configuration → Data Sources → Prometheus → Test

Docker Compose показывает ошибку при запуске

```
# Проверь .env файл
cat .env

# Если .env нет:
cp .env.example .env

# Проверь синтаксис docker-compose.yml
docker-compose config

# Попробуй с пересборкой
docker-compose up -d --build
```

---

8. ✅ Чек-лист перед защитой

Что проверить

- [ ] Docker Desktop установлен и работает
- [ ] Репозиторий склонирован: `git clone https://github.com/zaburdaev/my-devops-project.git`
- [ ] `.env` файл создан: `cp .env.example .env`
- [ ] `docker-compose up -d` запускается без ошибок

- [ ] Все 6 контейнеров в статусе Up/healthy: `docker-compose ps`
- [ ] Приложение открывается: http://localhost:5000
- [ ] Health check работает: http://localhost:5000/health
- [ ] Grafana открывается: http://localhost:3000 (учётные данные из `.env`)
- [ ] Дашборд показывает данные
- [ ] Тесты проходят: `python -m pytest tests/ -v`
- [ ] GitHub Actions — последний запуск зелёный (тесты + сборка)
- [ ] Ты понимаешь, что делает каждый файл (перечитай это руководство)

Что подготовить

- [ ] Ноутбук с работающим Docker и запущенным проектом
- [ ] Открытые вкладки в браузере:
- GitHub репозиторий
- GitHub Actions (вкладка Actions)
- Приложение (localhost:5000)
- Grafana (localhost:3000)
- Prometheus (localhost:9090)
- [ ] Терминал с открытой папкой проекта
- [ ] Прочитанное руководство (это!)

Что взять с собой

- [ ] Ноутбук + зарядку
- [ ] Стабильный интернет (для показа GitHub)
- [ ] Запущенный Docker Desktop
- [ ] Запущенные контейнеры (`docker-compose up -d`)
- [ ] Распечатку чек-листа (опционально)

Как отвечать на вопросы

«Что такое Docker?»

> «Docker — инструмент контейнеризации. Контейнер содержит приложение и все зависимости, что обеспечивает одинаковую работу в любой среде.»

«Зачем нужен CI/CD?»

> «CI/CD автоматизирует тестирование, сборку и деплой. При каждом push код автоматически проверяется и собирается Docker-образ.»

«Зачем нужен Kubernetes, если есть Docker Compose?»

> «Docker Compose подходит для разработки и небольших проектов. Kubernetes — для production: автоматическое масштабирование, self-healing, балансировка нагрузки.»

«Зачем Terraform, если можно создать сервер руками?»

> «Terraform — Infrastructure as Code. Код можно версионировать, воспроизводить и автоматизировать. Один `terraform apply` создаёт всю инфраструктуру.»

«Зачем Ansible, если есть Terraform?»

> «Terraform создаёт инфраструктуру (сервер), Ansible настраивает её (устанавливает ПО, деплоит приложение). Они дополняют друг друга.»

«Как работает мониторинг?»

> «Prometheus в текущей конфигурации собирает метрики с приложения каждые 60 секунд через endpoint /metrics. Grafana визуализирует эти метрики в виде графиков. Логи приложения доступны через docker compose logs.»

«Почему используется multi-stage build в Docker?»

> «Multi-stage build уменьшает размер финального образа. Первый этап устанавливает зависимости, второй копирует только нужное в чистый образ.»

«Что будет, если один контейнер упадёт?»

> «В Docker Compose — контейнер перезапустится (restart policy). В Kubernetes — система автоматически заменит упавший pod.»

> 💡 **Совет:** Перечитай это руководство за день до защиты. Запусти проект и пройди все шаги демонстрации. Уверенность — ключ к успеху! 🎯